Client / Freelancer
:ReviewView
:ReviewRoute
:ReviewController
:ReviewService
:ReviewRepo
:PrismaService
Database
1. Submit Delete Review
2. DELETE /api/backend/reviews/ :reviewId
3. DELETE /api/reviews/:reviewId; forward access cookie as Bearer
ref
3.1. Verify JWT
alt
[no Bearer token || verify failed || session is null]
2.1.1. Return 401 response
2.1.2. Display error message
[user is banned or deleted]
2.2.1. Return 403 response
2.2.2. Display error message
alt
[roleId missing OR permission denied]
3.2.1.1. 403 Forbidden
3.2.1.2. Return 403 response
3.2.1.3. Display permission error; stop
[role assigned AND permission allowed]
3.2.2.1. Continue to interceptor, pipes and handler
4. remove (userId, reviewId)
5. findById (reviewId)
6. review.findFirst: id, deletedAt=null; select reviewer, reviewee and response
7. Execute query
8. Return record or null
9. Return record or null
10. Return record or null
11. Handle lookup result
alt
[review absent]
11.1.1. 404 ReviewNotFoundException
11.1.2. Return HTTP error
11.1.3. Return error response
11.1.4. Display error message; stop this attempt
[lookup satisfies required condition]
11.2.1. Handle ownership result
alt
[review.reviewerId != userId]
11.2.1.1.1. 403 ReviewForbiddenException
11.2.1.1.2. Return HTTP error
11.2.1.1.3. Return error response
11.2.1.1.4. Display error message; stop this attempt
[current user owns record]
11.2.1.2.1. softDelete (reviewId)
11.2.1.2.2. review.update deletedAt=now
11.2.1.2.3. Execute query
11.2.1.2.4. Return write result or error
11.2.1.2.5. Return write result or error
11.2.1.2.6. Return write result or error
11.2.1.2.7. Handle persistence result
alt
[operation rejects]
11.2.1.2.7.1.1. Propagate original persistence error
11.2.1.2.7.1.2. Return HTTP error
11.2.1.2.7.1.3. Return error response
11.2.1.2.7.1.4. Display error message; stop this attempt
[operation resolves]
11.2.1.2.7.2.1. Return {message: ReviewMessage.DELETED}
11.2.1.2.7.2.2. Return serialized response
11.2.1.2.7.2.3. 200 OK: response data
11.2.1.2.7.2.4. Display result